

Module-Centric Repository Architecture: Best Practices Guide for AI

Document Purpose

This guide provides AI assistants with a comprehensive reference for understanding, creating, and refactoring repositories using module-centric architecture. It synthesizes proven patterns for organizing codebases where **modules are the atomic unit of ownership** rather than artifact types.

Target Audience: AI code assistants, automated refactoring tools, repository analyzers

Use Cases: Repository reorganization, new module creation, migration planning, architectural validation

Table of Contents

1. [Core Principles](#)
 2. [Module-Centric vs Artifact-Type Organization](#)
 3. [Module Structure Specification](#)
 4. [Module Classification System](#)
 5. [Identity and Naming Conventions](#)
 6. [Import Strategy Patterns](#)
 7. [Migration Methodology](#)
 8. [Validation and Quality Gates](#)
 9. [Required Documentation](#)
 10. [Anti-Patterns to Avoid](#)
-

Core Principles

The Eight Fundamental Rules

1. **Module Boundary Organization:** Organize by **module boundary**, NOT by artifact type
 - Group related functionality together
 - Each module owns its complete context
2. **Atomic Context Units:** Each module folder is the **atomic context unit** for AI tools
 - AI can load “one module at a time” as complete context
 - Minimizes cognitive load and context switching
3. **Co-located Artifacts:** Each module owns its **code, tests, schemas, docs, and state**
 - No separation between code and its tests
 - Documentation lives with the code it describes
4. **Identity-Based Organization:** Use ULID-style identity (or similar) to group related artifacts
 - Machine-verifiable relationships
 - Deterministic artifact discovery
5. **Incremental Migration:** Safe, incremental migration with no “big bang” rewrites
 - Preserve Git history
 - Keep old structure archived until validated
6. **Registry Abstraction:** Use ID + registry indirection, not hard-coded paths
 - Code knows IDs, registry knows paths
 - Paths can change without breaking imports
7. **AI-First Design:** Structure optimized for AI context loading and processing
 - Clear boundaries reduce hallucination
 - Complete module context in single directory
8. **Deterministic Execution:** Manifest-driven, with derived state from declarations
 - DAGs generated from dependencies, not hand-edited

- State transitions follow contracts

Module-Centric vs Artifact-Type Organization

Traditional Artifact-Type Structure (Anti-Pattern)

```
repository/
├─ src/                # All source code
│   ├─ core/
│   ├─ engine/
│   └─ utils/
├─ tests/              # All tests separated from code
│   ├─ test_core/
│   ├─ test_engine/
│   └─ test_utils/
├─ docs/               # All documentation separated
│   ├─ core/
│   ├─ engine/
│   └─ api/
├─ schemas/            # All schemas separated
│   ├─ database/
│   └─ api/
└─ config/             # All configs separated
    ├─ dev/
    └─ prod/
```

Problems with this approach:

- AI must traverse multiple directories to understand one feature
- Tests separated from code they test → drift and staleness
- Documentation scattered → outdated and contradictory
- No clear ownership boundaries
- Hard to load “just what I need” for AI context

Module-Centric Structure (Best Practice)

```
repository/
├─ modules/
│   ├─ core-engine/
│   │   └─ src/                # Module source code
```

```

| | | └─ tests/                # Tests for this module
| | | └─ docs/                # Module-specific docs
| | | └─ schemas/             # Module schemas
| | | └─ config/              # Module configs
| | | └─ .state/              # Module state
| | |   └─ current.json        # Current state
| | |     └─ module_tasks.dag.json
| | | └─ module.manifest.yaml  # Module contract
| | |   └─ __init__.py         # Import facade
| |
| └─ core-state/
|   └─ src/
|   └─ tests/
|   └─ docs/
|   └─ module.manifest.yaml
|
| └─ error-engine/
|   └─ src/
|   └─ tests/
|   └─ module.manifest.yaml
|
└─ docs/                        # ONLY global/cross-module docs
  └─ architecture/             # Architecture decisions
  └─ adr/                      # ADRs
  └─ reference/                 # Global references
|
└─ .state/                     # ONLY global state
  └─ dag/
    └─ repo_modules.dag.json    # Global dependency DAG
    └─ registry.json           # Module registry
|
└─ MODULES_INVENTORY.yaml       # Module catalog

```

Benefits:

- Single directory contains everything needed to understand a module
 - AI loads one directory = complete context
 - Tests adjacent to code → always in sync
 - Clear ownership and boundaries
 - Easy to validate module completeness
-

Module Structure Specification

Standard Module Layout

Every module **MUST** follow this structure:

```
modules/<module-id>/
├─ src/                                # Source code (REQUIRED)
│   ├── __init__.py
│   └─ <implementation files>
│
├─ tests/                              # Tests (REQUIRED)
│   ├── __init__.py
│   ├── unit/
│   └─ integration/
│
├─ docs/                               # Documentation (REQUIRED)
│   ├── README.md                     # Module overview
│   ├── API.md                        # Public interfaces
│   └─ ARCHITECTURE.md                # Internal design
│
├─ schemas/                           # Schemas (OPTIONAL)
│   ├── database/
│   └─ api/
│
├─ config/                             # Configuration (OPTIONAL)
│   ├── defaults.yaml
│   └─ profiles/
│
├─ examples/                           # Examples (OPTIONAL)
│   └─ usage_examples/
│
├─ scripts/                            # Module-specific scripts (OPTIONAL)
│   └─ maintenance/
│
├─ .state/                             # Module state (REQUIRED)
│   ├── current.json                  # Current module state
│   └─ module_tasks.dag.json          # Generated task DAG
│
├─ module.manifest.yaml                 # Module contract (REQUIRED)
└─ __init__.py                         # Import facade (REQUIRED)
```

Module Manifest Specification

Every module **MUST** have a `module.manifest.yaml` :

```
# Module identity
module_id: "core-engine"                # Unique identifier
module_kind: "PIPELINE_STAGE_MODULE"    # See classification
version: "1.2.0"                        # Semantic version
ulid_prefix: "01HQX7J9P2"              # Optional: ULID for file naming

# Module metadata
name: "Core Execution Engine"
description: "Orchestrates workflow execution and step processing"
owner: "platform-team"
status: "active"                        # active|deprecated|experimental

# Module architecture
layer: 2                                # Architectural layer (1-4)
dependencies:
  - module_id: "core-state"
    version: "^1.0.0"
    reason: "State persistence"
  - module_id: "error-engine"
    version: "^2.1.0"
    reason: "Error handling"

# Public interface
exports:
  - name: "ExecutionEngine"
    type: "class"
    path: "src/engine.py"
  - name: "execute_workflow"
    type: "function"
    path: "src/api.py"

# State contract
state_schema: "schemas/state.schema.json"
produces_events:
  - "workflow.started"
  - "workflow.completed"
  - "workflow.failed"

# Quality gates
test_coverage_minimum: 77
required_validators:
  - "python -m compileall src/ -q"
  - "pytest tests/ --cov=src --cov-report=term"
```

Module Classification System

Module Kind Taxonomy

Use this canonical classification for ALL modules:

1. PIPELINE_STAGE_MODULE

Purpose: Core sections of end-to-end processing pipeline

Examples: intake, planning, scheduling, execution, error-handling, state-persistence, reporting

Characteristics:

- Part of main data/workflow flow
- Has clear inputs and outputs
- Typically depends on lower layers

2. FEATURE_SERVICE_MODULE

Purpose: Cross-cutting capabilities used by multiple pipeline stages

Examples: tool-registry, pattern-engine, ai-logs-analyzer, template-system

Characteristics:

- Provides services to other modules
- Not part of main pipeline flow
- May be used by multiple stages

3. INTEGRATION_BRIDGE_MODULE

Purpose: Bridges to external systems and APIs

Examples: github-integration, jira-bridge, slack-notifier, external-cli-wrapper

Characteristics:

- Adapts external interfaces
- Handles authentication/authorization
- May have retry/circuit-breaker logic

4. INTERFACE_MODULE

Purpose: User-facing entry points

Examples: cli, gui, tui, web-api, rest-endpoints

Characteristics:

- Accepts user input
- Validates and routes requests
- Minimal business logic

5. REGISTRY_METADATA_MODULE

Purpose: ID systems, registries, artifact catalogs

Examples: doc-registry, module-registry, schema-catalog

Characteristics:

- Provides lookup/discovery
- Maintains indexes
- No business logic

6. INFRA_PLATFORM_MODULE

Purpose: Development infrastructure and tooling

Examples: ci-pipeline, environment-setup, build-tools, test-harness

Characteristics:

- Supports development process
- Not part of runtime system
- Configuration and automation

7. OBSERVABILITY_REPORTING_MODULE

Purpose: Logging, metrics, monitoring, reports

Examples: telemetry, metrics-collector, report-generator, log-aggregator

Characteristics:

- Collects and presents data
- Non-blocking
- Can be disabled without breaking core functions

8. GOVERNANCE_KNOWLEDGE_MODULE

Purpose: Documentation, standards, architectural decisions

Examples: adr-system, glossary, capability-catalog, best-practices

Characteristics:

- Contains knowledge artifacts
- No executable code (or minimal)
- Read-only at runtime

9. SANDBOX_EXPERIMENTAL_MODULE

Purpose: Experimental features and prototypes

Examples: feature-flags, alpha-features, experimental-apis

Characteristics:

- Unstable API
- May be removed without notice
- Not for production use

10. ARCHIVE_LEGACY_BUCKET

Purpose: Deprecated code kept for reference

Examples: old-api-v1, deprecated-flows, legacy-plugins

Characteristics:

- No longer maintained
- Kept for historical reference
- Should not be modified

Classification Decision Tree

Is it executable code?

└ NO → GOVERNANCE_KNOWLEDGE_MODULE

└ YES → Does it directly process user requests?

└ YES → INTERFACE_MODULE

└ NO → Is it part of the main processing pipeline?

└ YES → PIPELINE_STAGE_MODULE

└ NO → Does it integrate external systems?

└ YES → INTEGRATION_BRIDGE_MODULE

└ NO → Does it provide services to other modules?

└ YES → FEATURE_SERVICE_MODULE

```
└─ NO → Is it infrastructure/tooling?
    └─ YES → INFRA_PLATFORM_MODULE
    └─ NO → Is it observability?
        └─ YES → OBSERVABILITY_REPO
        └─ NO → REGISTRY_METADATA_
```

Identity and Naming Conventions

ULID-Based Identity System

ULID Format: 01HQX7J9P2MKN8VGQ3RWXYZ12 (26 characters)

- Timestamp-based (first 10 chars)
- Lexicographically sortable
- Globally unique
- Case-insensitive

File Naming with ULIDs

Pattern: {ULID_PREFIX}_descriptive-name.ext

Examples:

```
01HQX7J9P2_execution_engine.py
01HQX7J9P2_workflow_schema.json
01HQX7J9P2_user_guide.md
01HQX7J9P2_unit_tests.py
```

Benefits:

- Machine-verifiable grouping
- AI can identify related artifacts
- Deterministic discovery
- Chronological ordering

Module ID Conventions

Pattern: <domain>-<subdomain> (kebab-case)

Examples:

```
core-engine          # Good: clear, hierarchical
core-state           # Good: domain + responsibility
error-plugin-pylint  # Good: type + specific impl
aim-environment      # Good: namespace + scope

CoreEngine           # Bad: not kebab-case
core_engine          # Bad: snake_case reserved for files
engine               # Bad: too generic, no domain
```

Rules:

1. Use kebab-case (hyphens)
2. 2-3 segments maximum
3. Start with domain/namespace
4. End with specific responsibility
5. Avoid abbreviations unless standard (e.g., 'api', 'cli')

Registry ID Conventions

Pattern: {TYPE}_{DOMAIN}_{DESCRIPTOR}_{ULID}

Examples:

```
DOC_ARCH_MODULE_CENTRIC_01HGX7J9P2
PAT_EXEC_BATCH_CREATOR_01HGX8K3M4
CFG_AIM_PROFILE_DEFAULT_01HGX9L5N6
```

Import Strategy Patterns

The Python Numeric Filename Problem

Problem: Python cannot import modules starting with digits

```
# This FAILS:
from 01HGX7J9P2_engine import Engine # SyntaxError
```

Solution: Hybrid Import Pattern

Hybrid Import Pattern (Recommended)

Structure:

```
modules/core-engine/
├─ src/
│   ├── 01HQX7J9P2_execution_engine.py    # ULID filename (actual code)
│   ├── 01HQX7J9P2_workflow_processor.py
│   └─ __init__.py                        # Clean import facade
```

Implementation (__init__.py):

```
"""
Core Engine Module

Module ID: core-engine
ULID Prefix: 01HQX7J9P2
Layer: 2 (Domain)
"""

# Re-export from ULID files
from .01HQX7J9P2_execution_engine import (
    ExecutionEngine,
    execute_workflow,
    WorkflowStatus,
)

from .01HQX7J9P2_workflow_processor import (
    WorkflowProcessor,
    process_steps,
)

# Module metadata (for introspection)
__module_id__ = "core-engine"
__ulid_prefix__ = "01HQX7J9P2"
__layer__ = 2
__version__ = "1.2.0"

# Public API
__all__ = [
    "ExecutionEngine",
    "execute_workflow",
    "WorkflowStatus",
    "WorkflowProcessor",
    "process_steps",
]
```

Usage (clean imports):

```
# External code uses clean imports
from modules.core_engine import ExecutionEngine, execute_workflow

# Works perfectly - no ULID in import path
engine = ExecutionEngine()
result = execute_workflow(spec)
```

Benefits:

1. ULID files preserve machine-readable identity
2. `__init__.py` provides clean Python imports
3. AI can find related files via ULID prefix
4. No syntax errors
5. Stable public API

Import Rewriting Strategy

When migrating from old structure to module-centric:

Before (old artifact-type structure):

```
from core.engine import ExecutionEngine
from core.state import StateManager
from error.handlers import ErrorHandler
```

After (module-centric structure):

```
from modules.core_engine import ExecutionEngine
from modules.core_state import StateManager
from modules.error_engine import ErrorHandler
```

Automated Rewriting:

```
# Conversion rules
conversion_rules = {
    r"from core\.engine": "from modules.core_engine",
    r"from core\.state": "from modules.core_state",
    r"from error\.handlers": "from modules.error_engine",
}
```

```
# Apply to all Python files
for py_file in find_python_files():
    rewrite_imports(py_file, conversion_rules)
```

Migration Methodology

Phase-Based Migration Approach

Phase 0: Preparation and Analysis (1-2 hours)

Goals:

1. Analyze current repository structure
2. Create module inventory
3. Set up anti-pattern guards
4. Establish validation gates

Actions:

```
# Generate current state snapshot
tree -L 3 > repo_before.txt

# Analyze path usage
python scripts/analyze_paths.py > paths_summary.txt

# Create module inventory
python scripts/create_module_inventory.py

# Enable anti-pattern guards
python scripts/setup_guards.py --all
```

Outputs:

- MODULES_INVENTORY.yaml - Catalog of all modules
- paths_summary.txt - Current path usage analysis
- .execution/guards/ - Active anti-pattern guards
- Migration plan with phases

Phase 1: Create Module Skeletons (1 hour)

Goals:

1. Create empty module directories
2. Generate module manifests
3. Create `__init__.py` facades
4. No file moves yet

Actions:

```
# Create module structure
for module in $(yq '.modules[].module_id' MODULES_INVENTORY.yaml); do
    mkdir -p modules/$module/{src,tests,docs,schemas,config,.state}
    touch modules/$module/module.manifest.yaml
    touch modules/$module/__init__.py
done

# Generate manifests from templates
python scripts/generate_manifests.py --all

# Validate structure
python scripts/validate_module_structure.py --strict
```

Validation Gates:

```
# All modules have required directories
test $(find modules/ -name "module.manifest.yaml" | wc -l) -eq 34

# All manifests are valid YAML
yamllint modules/*/module.manifest.yaml

# All __init__.py files exist
test $(find modules/ -name "__init__.py" | wc -l) -eq 34
```

Phase 2: Update Registry Metadata (2 hours)

Goals:

1. Add `module_id` to all registry entries
2. Add `module_kind` classification

3. Maintain current paths (no moves yet)

4. Create module-to-artifact mapping

Actions:

```
# Extend registry schema
registry_schema = {
    "artifact_id": str,
    "artifact_kind": str,
    "path": str,
    "module_id": str,          # NEW
    "module_kind": str,       # NEW
    "version": str,
    "status": str,
}

# Backfill module metadata
for entry in registry:
    entry["module_id"] = infer_module_from_path(entry["path"])
    entry["module_kind"] = get_module_kind(entry["module_id"])
```

Validation Gates:

```
# All active entries have module_id
jq '.[] | select(.status == "active") | select(.module_id == null)' registry.json
# Should be empty

# All module_kinds are valid
jq '.[] | .module_kind' registry.json | sort -u
# Should only contain valid enum values
```

Phase 3: Module-by-Module Migration (3-10 hours)

Goals:

1. Move files one module at a time
2. Update registry paths
3. Rewrite imports
4. Validate after each module
5. Commit frequently

Pattern (for each module):


```

# 1. Move files for this module
MODULE="core-engine"
python scripts/migrate_module.py --module $MODULE --dry-run
python scripts/migrate_module.py --module $MODULE --execute

# 2. Update registry
python scripts/update_registry_paths.py --module $MODULE

# 3. Rewrite imports
python scripts/rewrite_imports.py --module $MODULE

# 4. Validate
python scripts/validate_module.py --module $MODULE --strict
python -m pytest modules/$MODULE/tests/ -v

# 5. Commit
git add modules/$MODULE/
git add registry.json
git commit -m "migrate: Move $MODULE to modules/"

```

Batch Processing (for speed):

```

# Process in batches of 10 modules
batches = chunk_modules(all_modules, size=10)

for batch in batches:
    for module in batch:
        migrate_module(module)

    # Validate entire batch
    validate_batch(batch)

    # Commit batch
    commit_batch(batch)

    # Checkpoint
    create_checkpoint(batch_id)

```

Phase 4: Archive Old Structure (1 hour)

Goals:

1. Move old artifact-type directories to archive
2. Update all documentation

3. Update index files

4. Final validation

Actions:

```
# Archive old structure
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
mkdir -p archive/structure_archived_${TIMESTAMP}

mv docs/ archive/structure_archived_${TIMESTAMP}/
mv tests/ archive/structure_archived_${TIMESTAMP}/
mv schemas/ archive/structure_archived_${TIMESTAMP}/
mv config/ archive/structure_archived_${TIMESTAMP}/

# Keep only global docs
mkdir docs/
mv archive/structure_archived_${TIMESTAMP}/docs/architecture docs/
mv archive/structure_archived_${TIMESTAMP}/docs/adr docs/

# Update CODEBASE_INDEX
python scripts/regenerate_codebase_index.py
```

Phase 5: Final Validation (1 hour)

Goals:

1. Run all tests
2. Validate all modules
3. Check all imports
4. Verify documentation
5. Create migration report

Validation Suite:

```
# 1. All modules compile
python -m compileall modules/ -q
echo $? # Must be 0

# 2. All tests pass
pytest modules/ -v --cov=modules --cov-report=term
# Coverage >= 77%
```

```
# 3. No ULID imports in code
grep -r "from.*01H" modules/*/src/ && echo "FAIL" || echo "PASS"

# 4. All manifests valid
python scripts/validate_all_manifests.py

# 5. DAG has no cycles
python scripts/validate_dag.py --check-cycles

# 6. Registry consistent
python scripts/validate_registry.py --check-paths
```

Rollback Strategy

Per-Phase Rollback:

```
# If Phase N fails, restore from checkpoint
git reset --hard $(cat .execution/checkpoints/phase_${(N-1)}_complete.commit)

# Verify restored
git log -1
```

Complete Rollback (emergency):

```
# Return to pre-migration state
git reset --hard $(cat .execution/migration_start.commit)

# Remove modules directory
rm -rf modules/

# Restore from archive if needed
cp -r archive/structure_archived_*/docs/ ./
cp -r archive/structure_archived_*/tests/ ./
```

Validation and Quality Gates

Automated Validation Gates

Every phase MUST pass these programmatic checks:

1. Structural Validation

```

def validate_module_structure(module_id):
    """Ensure module has required directories and files."""
    required = [
        f"modules/{module_id}/src",
        f"modules/{module_id}/tests",
        f"modules/{module_id}/docs",
        f"modules/{module_id}/.state",
        f"modules/{module_id}/module.manifest.yaml",
        f"modules/{module_id}/__init__.py",
    ]

    for path in required:
        assert os.path.exists(path), f"Missing: {path}"

    return True

```

2. Compilation Validation

```

# All Python files must compile without syntax errors
python -m compileall modules/ -q
EXIT_CODE=$?
if [ $EXIT_CODE -ne 0 ]; then
    echo "FAIL: Syntax errors detected"
    exit 1
fi

```

3. Import Validation

```

def validate_imports(module_id):
    """Ensure no ULID imports in public code."""
    src_files = glob(f"modules/{module_id}/src/**/*.py")

    for file_path in src_files:
        if file_path.endswith("__init__.py"):
            continue # __init__.py is allowed to import from ULID files

        with open(file_path) as f:
            content = f.read()
            if re.search(r'from\s+\.01H[A-Z0-9]+\_', content):
                raise ValidationError(f"{file_path} has ULID import")

    return True

```

4. Manifest Validation

```
def validate_manifest(module_id):
    """Validate module.manifest.yaml schema."""
    manifest_path = f"modules/{module_id}/module.manifest.yaml"

    with open(manifest_path) as f:
        manifest = yaml.safe_load(f)

    # Required fields
    required_fields = [
        "module_id",
        "module_kind",
        "version",
        "name",
        "description",
        "status",
        "layer",
    ]

    for field in required_fields:
        assert field in manifest, f"Missing field: {field}"

    # Valid module_kind
    valid_kinds = [
        "PIPELINE_STAGE_MODULE",
        "FEATURE_SERVICE_MODULE",
        "INTEGRATION_BRIDGE_MODULE",
        # ... other valid kinds
    ]
    assert manifest["module_kind"] in valid_kinds

    # Semantic version
    assert re.match(r'^\d+\.\d+\.\d+$', manifest["version"])

    return True
```

5. Dependency Validation

```
def validate_dependencies(module_id):
    """Check all dependencies exist and versions are compatible."""
    manifest = load_manifest(module_id)

    for dep in manifest.get("dependencies", []):
        dep_module_id = dep["module_id"]
```

```

    dep_version = dep["version"]

    # Dependency module exists
    assert os.path.exists(f"modules/{dep_module_id}")

    # Version compatibility
    dep_manifest = load_manifest(dep_module_id)
    assert version_compatible(dep_version, dep_manifest["version"])

    return True

```

6. Test Coverage Validation

```

# Run tests with coverage requirements
pytest modules/ \
    --cov=modules \
    --cov-report=term \
    --cov-fail-under=77 \
    -v

EXIT_CODE=$?
if [ $EXIT_CODE -ne 0 ]; then
    echo "FAIL: Test coverage below 77%"
    exit 1
fi

```

7. DAG Validation

```

def validate_dag_no_cycles():
    """Ensure dependency DAG has no cycles."""
    dag = load_repo_dag()

    # Topological sort will fail if cycles exist
    try:
        sorted_modules = topological_sort(dag)
    except CycleDetectedError as e:
        raise ValidationError(f"Cycle detected: {e.cycle}")

    return True

```

Ground Truth Validation Pattern

Use EXIT CODES for deterministic validation:

```
# GOOD: Uses exit code
python scripts/validate.py && echo "PASS" || echo "FAIL"

# BAD: Subjective evaluation
python scripts/validate.py
# Looks good!  <- Not deterministic
```

Required Documentation

Architecture Decision Records (ADRs)

Create these ADRs to document key decisions:

ADR-001: Module-Centric Architecture

```
# ADR-001: Module-Centric Architecture

## Status
ACCEPTED

## Context
Traditional artifact-type organization (separate `src/`, `tests/`, `docs/`)
creates problems for AI-assisted development:
- AI must traverse multiple directories for one feature
- Tests separated from code leads to drift
- No clear ownership boundaries

## Decision
Adopt module-centric architecture where each module owns all its artifacts.

## Consequences
Positive:
- AI can load complete context from one directory
- Co-located tests stay in sync with code
- Clear ownership and boundaries
- Easier to extract modules into separate repos

Negative:
- Requires migration of existing codebase
- Team must learn new structure
- Build tools may need updates
```

ADR-002: ULID Identity System

```
# ADR-002: ULID Identity System for Artifact Grouping

## Status
ACCEPTED

## Context
Need machine-verifiable way to identify related artifacts across a module.
Git SHA is post-hoc, filenames alone insufficient.

## Decision
Use ULID (Universally Unique Lexicographically Sortable Identifier) prefixes
in filenames to group related artifacts.

## Implementation
- Format: `{ULID_PREFIX}_descriptive-name.ext`
- Example: `01HQX7J9P2_execution_engine.py`
- ULID stored in `module.manifest.yaml`

## Consequences
Positive:
- Machine can verify artifact relationships
- Chronological ordering from timestamp
- Works across file types (code, docs, schemas)

Negative:
- Longer filenames
- Requires tooling to generate/validate ULIDs
```

ADR-003: Hybrid Import Strategy

```
# ADR-003: Hybrid Import Strategy for Python

## Status
ACCEPTED

## Context
Python cannot import modules starting with digits, but ULID prefixes
start with digits. Need both ULID identity and clean imports.

## Decision
Use hybrid approach:
- Actual code files use ULID prefixes: `01HQX7J9P2_engine.py`
- `__init__.py` re-exports with clean names
```



```
- External code imports from `__init__.py`

## Implementation
```python
modules/core_engine/__init__.py
from .01HQX7J9P2_engine import ExecutionEngine
__all__ = ["ExecutionEngine"]
```

## Consequences

### Positive:

- Preserves ULID identity
- Clean Python imports work
- No syntax errors

### Negative:

- Requires maintaining `__init__.py`
- Import facade adds indirection

```
Required Module Documentation
```

Each module MUST have these documentation files:

```
modules/{module-id}/docs/README.md
```

```
```markdown
```

```
# {Module Name}
```

```
**Module ID**: `{module-id}`
```

```
**Module Kind**: `{MODULE_KIND}`
```

```
**Layer**: {layer_number}
```

```
**Version**: {version}
```

```
## Purpose
```

Brief description of what this module does and why it exists.

```
## Public API
```

```
### Exported Functions
```

```

- `function_name(args) -> return_type`: Description

### Exported Classes

- `ClassName`: Description

## Dependencies

- `other-module-id` (version constraint): Why needed

## Usage Examples

```python
from modules.{module_id} import SomeClass

obj = SomeClass()
result = obj.do_something()

```

## State Contract

What state this module maintains and how it transitions.

## Testing

How to run this module's tests:

```

pytest modules/{module-id}/tests/ -v

modules/{module-id}/docs/ARCHITECTURE.md

```markdown
# {Module Name} - Internal Architecture

## Component Structure

[Diagram or description of internal components]

## Data Flow

[How data moves through the module]

## Design Decisions

```

Why X instead of Y?

Rationale for key design choices.

Extension Points

How to extend or customize this module.

Internals

Implementation details for maintainers.

Global Documentation Structure

```
docs/
├── architecture/
│   ├── ADR-001-module-centric.md
│   ├── ADR-002-ulid-identity.md
│   ├── ADR-003-hybrid-imports.md
│   ├── SYSTEM_ARCHITECTURE.md
│   └── MODULE_DEPENDENCY_GRAPH.md
│
├── developer/
│   ├── MODULE_CREATION_GUIDE.md
│   ├── TESTING_STRATEGY.md
│   └── IMPORT_PATTERNS.md
│
├── migration/
│   ├── MIGRATION_RUNBOOK.md
│   ├── ROLLBACK_PROCEEDURES.md
│   └── VALIDATION_CHECKLIST.md
│
├── reference/
│   ├── MODULE_MANIFEST_SPEC.md
│   ├── ULID_CONVENTIONS.md
│   └── MODULE_KINDS_REFERENCE.md
│
└── operations/
    ├── MODULE_STATE_MANAGEMENT.md
    ├── DAG_REFRESH_WORKFLOW.md
    └── HEALTH_MONITORING.md
```

Anti-Patterns to Avoid

Critical Anti-Patterns (Tier 1)

1. Hallucination of Success

Problem: Claiming operations succeeded without verification

Cost: 12 hours wasted debugging phantom completions

Anti-Pattern:

```
# Move files
os.rename("old/path", "new/path")
print("✅ Migration complete!") # NO VERIFICATION
```

Correct Pattern:

```
# Move files
os.rename("old/path", "new/path")

# VERIFY
assert os.path.exists("new/path"), "Migration failed: target missing"
assert not os.path.exists("old/path"), "Migration failed: source still exists"

print("✅ Migration verified")
```

2. Incomplete Implementation

Problem: Partial execution without completing all steps

Cost: 5 hours wasted on broken state

Anti-Pattern:

```
# Migrate module
move_files(module)
# Forgot to update registry
# Forgot to rewrite imports
# Forgot to update docs
```

Correct Pattern:

```
# Complete migration checklist
def migrate_module(module_id):
```

```

move_files(module_id)
update_registry(module_id)
rewrite_imports(module_id)
update_docs(module_id)
validate_module(module_id) # Catches incomplete work

```

3. Silent Failures

Problem: Errors ignored or not checked

Cost: 4 hours debugging cascading failures

Anti-Pattern:

```

try:
    critical_operation()
except Exception:
    pass # Silent failure

```

Correct Pattern:

```

try:
    critical_operation()
except Exception as e:
    logger.error(f"Critical operation failed: {e}")
    raise # Re-raise for caller to handle

```

High Priority Anti-Patterns (Tier 2)

4. Framework Over-Engineering

Problem: Building generic frameworks instead of solving immediate problem

Cost: 10 hours building unused abstractions

Anti-Pattern:

```

# Building a generic module migration framework with plugins,
# hooks, event system, and extensibility for future unknown needs
class PluginableModuleMigrator:
    def __init__(self):
        self.plugins = []
        self.hooks = {}
        # ... 500 lines of abstraction

```

Correct Pattern:

```
# Simple, direct solution
def migrate_module(module_id):
    """Migrate one module. Repeat for each module."""
    move_files(module_id)
    update_registry(module_id)
    validate(module_id)
```

5. Planning Loop Trap

Problem: Endless planning without execution

Cost: 16 hours planning, 0 hours executing

Anti-Pattern:

```
Iteration 1: Create detailed plan
Iteration 2: Refine plan with edge cases
Iteration 3: Add risk mitigation to plan
Iteration 4: Consider alternative approaches
Iteration 5: Optimize plan structure
[Never actually execute]
```

Correct Pattern:

```
Iteration 1: Create minimal viable plan
Iteration 2: Execute plan
[Stop planning, start doing]
```

6. Test-Code Mismatch

Problem: Tests test old behavior after code changes

Cost: 6 hours debugging "passing" tests that test wrong thing

Anti-Pattern:

```
# Code changed to new structure
from modules.core_engine import Engine

# Tests still use old structure
from core.engine import Engine # Imports from archive!
```

Correct Pattern:

```
# Update tests simultaneously with code
from modules.core_engine import Engine # Matches current code
```

Medium Priority Anti-Patterns (Tier 3)

7. Configuration Drift

Problem: Configuration files inconsistent with code

Cost: 3 hours debugging why “correct” code fails

Anti-Pattern:

```
# pyproject.toml still references old paths
[tool.pytest.ini_options]
testpaths = ["tests"] # Old location

# But tests are now in modules/*/tests/
```

Correct Pattern:

```
# Updated configuration
[tool.pytest.ini_options]
testpaths = ["modules"]
python_files = ["test_*.py", "*_test.py"]
```

8. Partial Success Amnesia

Problem: Forgetting what was completed in previous execution

Cost: 12 hours re-doing already completed work

Anti-Pattern:

```
# Run migration script
python migrate.py # Migrates modules 1-10

# Script crashes

# Re-run from beginning
python migrate.py # Re-migrates modules 1-10 again!
```

Correct Pattern:

```

# Create checkpoints
migrate_modules(1-10)
create_checkpoint("batch_1_complete")

# Script crashes after batch 2

# Resume from checkpoint
resume_from_checkpoint("batch_1_complete")
migrate_modules(11-20) # Skips already completed

```

9. Module Integration Gap

Problem: Modules work in isolation but fail when integrated

Cost: 2 hours fixing integration issues

Anti-Pattern:

```

# Test only individual modules
pytest modules/core_engine/tests/ # Passes
pytest modules/core_state/tests/  # Passes

# Integration never tested
# Fails in production when modules interact

```

Correct Pattern:

```

# Test both isolation and integration
pytest modules/core_engine/tests/ # Unit tests
pytest modules/core_state/tests/  # Unit tests
pytest tests/integration/         # Integration tests

```

Anti-Pattern Guards (Automated Prevention)

Implement these automated checks:

```

# Guard #1: Programmatic Verification
def validate_with_exit_code():
    """Always return exit code for programmatic checking."""
    success = perform_operation()
    sys.exit(0 if success else 1)

# Guard #2: Complete Checklist
def require_all_steps():
    """Enforce completion of all steps."""

```



```

steps = [move, update, rewrite, validate]
for step in steps:
    result = step()
    assert result.success, f"{step} failed"

# Guard #3: Checkpoint Progress
def checkpoint_after_batch():
    """Save progress after each batch."""
    process_batch()
    save_checkpoint(batch_id, timestamp, state)

# Guard #4: Validate Before Commit
def pre_commit_validation():
    """Run full validation before committing."""
    if not all_tests_pass():
        raise PreCommitError("Tests must pass before commit")

```

AI Assistant Guidelines

Context Loading Strategy

When asked to work on a module-centric repository:

1. **Identify the module:** Determine which module the request concerns
2. **Load module context:** Read entire module directory (not just code)
3. **Check manifest:** Understand module dependencies and contracts
4. **Review related modules:** Load dependencies if needed
5. **Maintain module boundaries:** Don't mix concerns across modules

Example AI Workflow

User Request: "Add error handling to the execution engine"

AI Response Pattern:

1. Identify module: core-engine
2. Load context:
 - Read modules/core-engine/docs/README.md
 - Read modules/core-engine/module.manifest.yaml
 - Check dependencies: [core-state, error-engine]
 - Load modules/error-engine/docs/API.md (for error types)

3. Analyze change scope:
 - Modify: `modules/core-engine/src/01HGX7J9P2_engine.py`
 - Update: `modules/core-engine/tests/test_error_handling.py`
 - Document: `modules/core-engine/docs/ERROR_HANDLING.md`
4. Validate:
 - Run module tests
 - Check imports still work
 - Verify manifest still accurate

Decision Hierarchy for AI

When making architectural decisions:

1. **Module manifest is truth:** If manifest says it, honor it
 2. **Registry over filesystem:** Use registry to find artifacts, not glob
 3. **Validate everything:** Never assume, always verify
 4. **Fail explicitly:** Better to error than produce subtle bugs
 5. **Document deviations:** If you must deviate from pattern, document why
-

Summary Checklist

Use this checklist when creating or refactoring to module-centric architecture:

Module Structure

- Each module has `src/`, `tests/`, `docs/`, `.state/`
- Each module has `module.manifest.yaml`
- Each module has `__init__.py` import facade
- ULID prefixes used consistently (if applicable)
- No artifact-type directories at root (except global)

Module Metadata

- `module_id` is kebab-case, clear, unique
- `module_kind` from canonical taxonomy
- `version` is semantic (X.Y.Z)

- `layer` number assigned (1-4)
- Dependencies declared in manifest

Import Strategy

- ULID files in `src/` , clean re-exports in `__init__.py`
- No ULID imports in public code
- All imports validated (compile test)
- Import paths updated from old structure

Validation

- All modules compile without errors
- All tests pass (unit + integration)
- Test coverage $\geq 77\%$
- Dependency DAG has no cycles
- Registry paths match filesystem

Documentation

- Module README.md exists and complete
- Architecture ADRs created
- Migration procedures documented
- Rollback procedures tested
- API documentation current

Safety

- Old structure archived, not deleted
 - Git history preserved
 - Checkpoints created during migration
 - Rollback tested and works
 - All anti-pattern guards enabled
-

Conclusion

Module-centric architecture provides a deterministic, AI-friendly structure that:

- Reduces cognitive load through clear boundaries
- Enables AI to load complete context from single directory
- Maintains co-location of related artifacts
- Supports safe, incremental migration
- Enforces contracts through manifests
- Validates structure programmatically

Follow these best practices to create repositories that are maintainable, understandable, and optimized for AI-assisted development.